RAG for SmartGrid AI: Knowledge Retrieval and Grounded Explanations

A technical reference for integrating ML outputs with document evidence

Document purpose: This document is prepared as a searchable knowledge-base source for a SmartGrid AI RAG prototype. It is an educational synthesis, not an official ERCOT, utility, or government publication.

What RAG does

Retrieval-Augmented Generation (RAG) combines information retrieval with language generation. Instead of asking a language model to answer only from its general learned knowledge, the application first searches a supplied knowledge base and then provides relevant passages as context for the response.

In SmartGrid AI, the ML model estimates demand or grid-stress risk. RAG retrieves relevant material about grid operations, weather impacts, demand response, reliability, and preparedness. The language model uses both the structured prediction and retrieved passages to produce a readable explanation.

Document ingestion

Documents are loaded from PDFs, text files, or other supported formats. Text extraction should preserve headings and page references where possible. Scanned PDFs may require OCR. Extraction quality should be checked because missing tables, broken reading order, or corrupted characters can reduce retrieval quality.

Documents should be chunked into manageable passages. Chunks that are too large may contain unrelated topics; chunks that are too small may lose context. Store metadata such as filename, page number, section title, publication date, and source URL with each chunk.

Indexing and retrieval

TF-IDF represents text using term importance and can work as a lightweight baseline for keyword-oriented retrieval. It is relatively simple to implement and does not require a large embedding model. However, it may miss semantic similarity when a query uses different words from the source passage.

Dense embeddings represent semantic meaning and may retrieve paraphrases more effectively, but they require an embedding model and additional compute or service access. A vector database or nearest-neighbor index can help search large collections efficiently. The choice should match the project's compute, privacy, and deployment constraints.

Evaluate retrieval with representative questions. Inspect whether the top retrieved chunks actually contain the evidence needed to answer. Tune chunk size, overlap, text cleaning, top-k, and metadata filters based on observed failures.

Generation and grounding

Pass the user question, structured ML prediction, and retrieved passages to the LLM. Instruct it to answer only from the provided evidence for factual claims, distinguish model estimates from documented facts, and state when the documents do not contain enough information.

Ask the LLM to cite document names or page numbers in its answer. Keep retrieved passages available for audit. If the retrieval results are weak or irrelevant, the system should say that evidence is insufficient rather than inventing a cause.

Suggested integration contract

An ML service can return a structured object containing prediction timestamp, forecast horizon, predicted demand, demand units, grid-stress class, model score, and relevant feature values. The RAG service should accept this object and build a focused retrieval query.

An example response object can include: summary, risk_label, model_score, forecast_period, possible_contributing_factors, consumer_guidance, evidence_sources, and limitations. Do not label a model score as a calibrated probability unless the ML team has evaluated calibration.

Testing and evaluation

Test retrieval separately from generation. Retrieval tests ask whether the right source passages are returned. Generation tests ask whether the answer is supported by those passages, avoids unsupported claims, and accurately reflects the ML input.

Build a small evaluation set of realistic questions, expected source documents, and required facts. Include questions whose answers are absent from the knowledge base to test abstention. Log query, retrieved chunk identifiers, model input, output, and errors while protecting sensitive information.

Key takeaways

- RAG retrieves evidence; the LLM turns evidence and model outputs into readable explanations.
- TF-IDF is a useful lightweight baseline but is less semantic than dense embeddings.
- Keep source metadata and citations with every chunk.
- Evaluate retrieval and generation separately, and make the system abstain when evidence is missing.

References and further reading

• Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks — https://arxiv.org/abs/2005.11401

• ERCOT Grid Information — https://www.ercot.com/gridinfo

Prepared 9 November 2026. Verify current operational information with the linked official sources before making real-world decisions.